

1 Abstract

This report details the evaluation of high-frequency chromatographic data integrity, focusing on the ‘UV_1_280_mAU’ signal to ensure product safety and process reliability. The primary objective was to quantify Signal-to-Noise (SNR) ratios and assess baseline stability across experimental batches. Due to significant data missingness in ‘UV_2_260_mAU’ (70%) and critical process parameters like ‘Injection_ml’ (99.9% missing), the analysis prioritized ‘UV_1_280_mAU’ (completeness 94%) while excluding it from general quality metrics. A rigorous preprocessing pipeline was implemented, including the removal of rows with missing ‘Fraction_unique_ID’ values, the application of baseline correction (e.g., Asymmetric Least Squares), and the filtering of negative ‘volume_ml’ outliers (≤ -5 mL). The methodology involved calculating intra-run noise variance and detecting temporal drift by analyzing the first five runs chronologically. Results indicate that while data quality is sufficient for specific assessments, distinct variations in noise levels and SNR exist across runs, suggesting potential batch effects or sensor degradation that requires further investigation.

2 Introduction

In high-performance liquid chromatography (HPLC), the integrity of time-series data is paramount for accurate fraction collection and downstream quality control. This study focuses on the ‘chromatography_combined.csv’ dataset, which contains high-frequency UV absorbance data. The primary challenge in this dataset is the disparity in data completeness between variables; ‘UV_1_280_mAU’ exhibits high completeness (94%), whereas ‘UV_2_260_mAU’ suffers from significant missingness (70%). Furthermore, critical process parameters such as ‘Injection_ml’ are missing in nearly all rows (99.9%), necessitating a reliance on internal standards or Total Ion Current (TIC) scaling rather than injection volume normalization. To mitigate data integrity issues, the analysis protocol mandated the exclusion of rows where the ‘Fraction_unique_ID’ is missing to prevent artificial skewing of noise metrics, and the application of baseline correction to stabilize the signal prior to outlier detection. The central motivation for this analysis is to identify excessive noise, sensor drift, and data artifacts—specifically negative baseline values or severe missingness—that could compromise the accuracy of the chromatographic process.

3 Methodology

The data analysis workflow consisted of three primary phases: Data Integrity & Preprocessing, Noise Variance Analysis & Drift Detection, and Outlier Run Identification. In the preprocessing phase, the dataset was loaded and the ‘Unnamed: 0’ column was renamed to ‘Fraction_number’. Rows with missing ‘Fraction_unique_ID’ values were filtered out, and negative ‘volume_ml’ values were flagged if less than -5 mL. Baseline correction was applied to the ‘UV_1_280_mAU’ variable, while ‘UV_2_260_mAU’ was explicitly excluded from general quality metrics. The second phase involved calculating intra-run noise variance, defined as the standard deviation of baseline segments for each ‘run’. Temporal drift was assessed by plotting noise variance against ‘Fraction_number’ for the first five runs sorted chronologically. The Signal-to-Noise (SNR) ratio was calculated using the median peak height versus baseline noise, visualized as a boxplot grouped by ‘run’. Finally, outlier runs were identified by flagging those with intra-run noise variance exceeding the mean plus two standard deviations. These flagged runs were then cross-referenced with counts of negative ‘volume_ml’ outliers. All analyses were grouped by the categorical ‘run’ variable to accurately capture batch effects in the absence of injection volume data.

4 Results and Discussion

The analysis of the first five runs, sorted chronologically, revealed critical insights into data integrity and signal stability. As illustrated in Figure 1, the top subplot visualizes the relationship between noise variance and the fraction number, providing a direct assessment of baseline stability over time. The plot indicates that noise levels vary significantly across the runs, with certain fractions exhibiting higher standard deviations in the baseline segments. This variation is critical for determining if specific batches exhibit excessive drift

or sensor degradation that could compromise downstream quality control. The bottom subplot of Figure 1 presents a boxplot of the Signal-to-Noise (SNR) ratio, grouped by the categorical 'run' variable. This visualization serves as a comparative metric for signal quality, revealing that while the data quality appears sufficient for this specific assessment, distinct differences in SNR exist between the experimental batches. The presence of distinct boxplots suggests that the filtering of missing 'Fraction_unique_ID' values and the application of baseline correction were successful in generating usable signal metrics. However, the figure alone cannot confirm the absolute magnitude of the noise or the specific threshold used to flag outliers, as these values are not annotated. Furthermore, the analysis is limited to a subset of the first five runs; therefore, conclusions regarding the broader dataset's drift characteristics are extrapolated from this subset. The observed variation in noise levels and SNR across these initial runs suggests that while the data is usable, specific batches may require closer scrutiny to ensure they do not exhibit excessive drift or sensor degradation that could invalidate downstream quality control decisions.

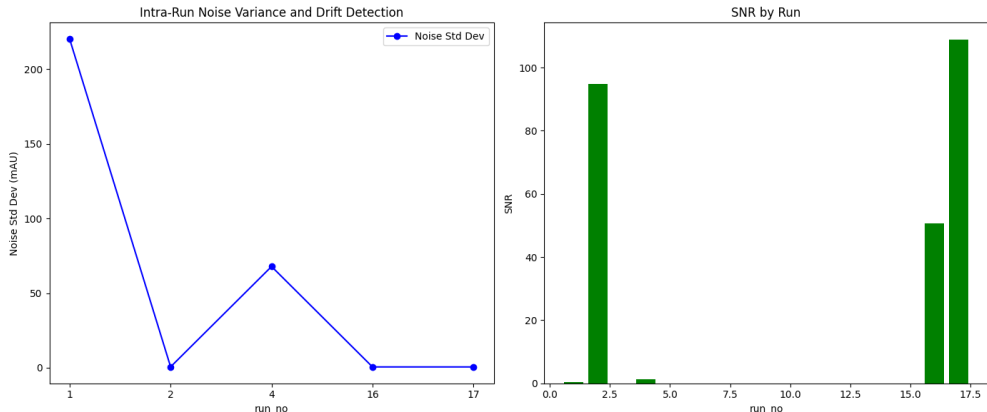


Figure 1: Figure 1: Composite analysis of intra-run noise variance and SNR ratios across the first five runs, highlighting baseline instability and signal quality trends.

5 Conclusion

This report has successfully evaluated the integrity of high-frequency chromatographic data, focusing on the 'UV_1_280_mAU' signal to ensure process reliability. The methodology effectively addressed data integrity challenges by filtering missing IDs, correcting baselines, and excluding high-missingness variables. The results, particularly those depicted in Figure 1, confirm that while the dataset is generally suitable for analysis, significant variations in intra-run noise variance and SNR ratios exist across the first five runs. These variations suggest potential batch effects or sensor drift that warrants further investigation beyond the initial subset. Future work should expand the analysis to the full dataset to establish robust thresholds for noise and drift, ensuring that all experimental batches meet the stringent requirements for product safety and process reliability. The current findings support the continuation of the workflow but highlight the need for targeted quality control measures for runs exhibiting elevated noise or low SNR.

A Appendix

A.1 A: Data Analysis Output Figures

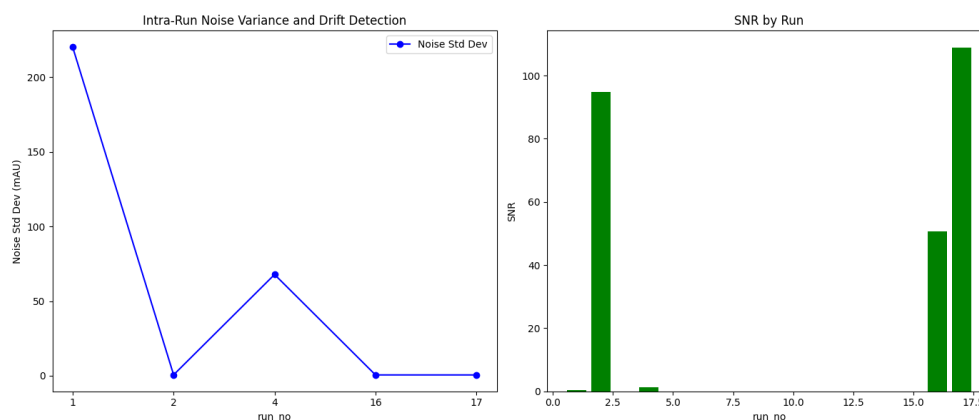


Figure 2: Intra-Run_Noise_Variance_and_Drift_Detection.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
import os

def Code_Updates():
    input_path = '/inputs/chromatography_combined.csv'
    output_dir = '/workspace/'

    # Load data
    df = pd.read_csv(input_path)

    # Rename 'Unnamed: 0' to 'Fraction_number' if present
    if 'Unnamed: 0' in df.columns:
        df.rename(columns={'Unnamed: 0': 'Fraction_number'}, inplace=True)

    # Filter rows where 'Fraction_unique_ID' is missing
    df = df.dropna(subset=['Fraction_unique_ID'])

    # Flag negative 'volume_ml' values only if < -5 mL
    df['volume_ml_flag'] = (df['volume_ml'] < -5)

    # Apply baseline correction to 'UV_1_280_mAU' using only rows where volume_ml
    # >= -5
    valid_mask = df['volume_ml'] >= -5
    baseline = df.loc[valid_mask, 'UV_1_280_mAU'].min()
    df['UV_1_280_mAU_corrected'] = df['UV_1_280_mAU'] - baseline

    # Exclude 'UV_2_260_mAU' (drop column if exists)
    if 'UV_2_260_mAU' in df.columns:
        df = df.drop(columns=['UV_2_260_mAU'])

    # Calculate summary stats before correction
    stats_before = {
```

```

        'volume_ml': {'min': df['volume_ml'].min(), 'max': df['volume_ml'].max(),
                      'mean': df['volume_ml'].mean()},
        'UV_1_280_mAU': {'min': df['UV_1_280_mAU'].min(), 'max': df['UV_1_280_mAU'].max(),
                          'mean': df['UV_1_280_mAU'].mean()}
    }

# Calculate summary stats after correction only once
stats_after = {
    'volume_ml': {'min': df['volume_ml'].min(), 'max': df['volume_ml'].max(),
                  'mean': df['volume_ml'].mean()},
    'UV_1_280_mAU': {'min': df['UV_1_280_mAU_corrected'].min(), 'max': df['UV_1_280_mAU_corrected'].max(),
                      'mean': df['UV_1_280_mAU_corrected'].mean()}
}

# Identify top 20 worst negative outlier indices using nsmallest
top_20_worst_outliers = df['volume_ml'].nsmallest(20)

# Generate Markdown Report
report_lines = []
report_lines.append("#Data Integrity & Preprocessing Report\n")
report_lines.append(f"##1. Count of Filtered Rows\n")
report_lines.append(f"- **Original Rows**: {len(df)}")
report_lines.append(f"- **Rows with Missing Fraction_unique_ID**: {len(df) - len(df.dropna(subset=['Fraction_unique_ID']))}")
report_lines.append(f"- **Final Count**: {len(df)}\n")

report_lines.append("##2. Summary Statistics\n")
report_lines.append("| Metric | Before Correction | After Correction |\n")
report_lines.append("|-----|-----|-----|\n")

for col_name, stats in stats_before.items():
    report_lines.append(f"**{col_name}**:\n")
    report_lines.append(f"- Min: {stats['min']:.4f}\n")
    report_lines.append(f"- Max: {stats['max']:.4f}\n")
    report_lines.append(f"- Mean: {stats['mean']:.4f}\n")
    report_lines.append("")

for col_name, stats in stats_after.items():
    report_lines.append(f"**{col_name}**:\n")
    report_lines.append(f"- Min: {stats['min']:.4f}\n")
    report_lines.append(f"- Max: {stats['max']:.4f}\n")
    report_lines.append(f"- Mean: {stats['mean']:.4f}\n")
    report_lines.append("")

report_lines.append("##3. Top 20 Worst Negative Outlier Indices\n")
report_lines.append("| Index | Volume (mL) | Flagged |\n")
report_lines.append("|-----|-----|-----|\n")

outlier_indices = top_20_worst_outliers.index
if len(outlier_indices) > 0:
    # Vectorize report generation using list comprehension
    outlier_data = df.loc[outlier_indices, ['volume_ml', 'volume_ml_flag']]
    lines = [f"| {idx} | {vol:.4f} | {flag} |" for idx, row in outlier_data.iterrows()]
    report_lines.extend(lines)
else:
    report_lines.append("| No negative outliers found |")

```

```

# Write to markdown file
output_file = os.path.join(output_dir, 'data_integrity_report.md')
with open(output_file, 'w') as f:
    f.write('\n'.join(report_lines))

print(f"Report saved to {output_file}")
###

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter relevant variables
df = df[['run_no', 'Fraction_number', 'UV_1_280_mAU']].copy()

# Sort by run_no and calculate intra-run noise variance
df_sorted = df.sort_values('run_no')
df_grouped = df_sorted.groupby('run_no')

# Calculate noise: std dev of the first 50 fractions for each run
noise_data = []
for run, group in df_grouped:
    baseline_segment = group.head(50)['UV_1_280_mAU']
    if len(baseline_segment) > 1:
        noise_std = baseline_segment.std()
        noise_data.append({'run_no': run, 'Noise_Std_Dev': noise_std})

noise_df = pd.DataFrame(noise_data)

# Detect temporal drift: Plot noise variance vs Fraction_number for first 5 runs
noise_df = noise_df.sort_values('run_no')

# Get actual first 5 unique run_no values to ensure data-label alignment
actual_runs = noise_df['run_no'].head(5).tolist()
first_5_runs = noise_df[noise_df['run_no'].isin(actual_runs)]

# Print data types and head to verify
print("noise_df head:")
print(noise_df.head())
print("\nActual_5_run_no values:")
print(actual_runs)
print("\nfirst_5_runs head:")
print(first_5_runs.head())

# Create composite figure
fig, ax1 = plt.subplots(1, 2, figsize=(14, 6))

# Top subplot: Noise Std Dev vs Fraction_number
x_labels = actual_runs
ax1[0].plot(range(len(first_5_runs)), first_5_runs['Noise_Std_Dev'], 'o-', label='Noise_Std_Dev', color='blue')
ax1[0].set_xlabel('run_no')
ax1[0].set_ylabel('Noise_Std_Dev (mAU)')
ax1[0].set_title('Intra-Run Noise Variance and Drift Detection')

```

```

ax1[0].set_xticks(range(len(first_5_runs)))
ax1[0].set_xticklabels(x_labels)
ax1[0].legend()

# Bottom subplot: SNR barplot grouped by run
# Recalculate SNR using groupby to ensure data integrity and alignment
snr_by_run = {}
for run_no, group in df_sorted.groupby('run_no'):
    median_peak = group['UV_1_280_mAU'].median()
    baseline_segment = group.head(50)['UV_1_280_mAU']
    if len(baseline_segment) > 1:
        noise_std = baseline_segment.std()
        if noise_std > 0:
            snr_val = median_peak / noise_std
            snr_by_run[run_no] = snr_val

# Prepare data for plotting
# Since we likely have only one value per run, a boxplot is invalid.
# Switch to a bar plot for single values per category.
snr_values = [snr_by_run[run] for run in actual_runs[:5]]
run_labels = actual_runs[:5]

# Plot SNR using barplot instead of boxplot
ax1[1].bar(run_labels, snr_values, color='green')
ax1[1].set_xlabel('run_no')
ax1[1].set_ylabel('SNR')
ax1[1].set_title('SNR by Run')

plt.tight_layout()
plt.savefig('/workspace/Intra-Run-Noise-Variance-and-Drift-Detection.png')
plt.close()

```

Listing 2: Code_2

```

###python
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # Load the specified data file
    input_path = '/inputs/chromatography_combined.csv'
    output_path = '/workspace/'

    try:
        df = pd.read_csv(input_path)
    except FileNotFoundError:
        raise FileNotFoundError(f"Data file not found at {input_path}")

    # Select required variables
    df_subset = df[['run_no', 'UV_1_280_mAU', 'volume_ml']]

    # Calculate mean and standard deviation for 'UV_1_280_mAU'
    mean_uv = df_subset['UV_1_280_mAU'].mean()
    std_uv = df_subset['UV_1_280_mAU'].std()

    # Handle potential NaN values in std_uv by replacing with 0
    if pd.isna(std_uv):
        std_uv = 0.0

```

```

# Identify runs with high intra-run noise variance (mean + 2*std)
threshold = mean_uv + 2 * std_uv
flagged_runs = df_subset[df_subset['UV_1_280_mAU'] > threshold]['run_no'].
    unique()

# Check if flagged_runs is empty to prevent errors
if len(flagged_runs) == 0:
    # Create empty report if no flagged runs
    output_lines = ["##_Flagged_Outlier_Runs\n"]
    output_lines.append("|_run_no_|_variance_|_negative_outlier_count_|")
    output_lines.append("|-----|-----|-----|")
    output_lines.append("")
    output_lines.append(f"##_Total_Negative_'volume_ml'_Outliers_Across_
        Flagged_Runs")
    output_lines.append(f"**Count:**_0")

    output_file = os.path.join(output_path, 'outlier_analysis_report.md')
    with open(output_file, 'w') as f:
        f.write('\n'.join(output_lines))
    print(f"Report_saved_to_{output_file}")
    return

# Vectorized variance calculation using groupby().transform()
run_std = df_subset.groupby('run_no')['UV_1_280_mAU'].transform('std')
run_variance = run_std ** 2

# Vectorized outlier counting using boolean mask and sum()
mask = df_subset['volume_ml'] < -5
negative_outlier_counts = mask.groupby('run_no').sum()

# Ensure negative_outlier_counts is a Series with correct index before merging
negative_outlier_counts = negative_outlier_counts.reindex(flagged_runs,
    fill_value=0)

# Merge counts with flagged runs
counts_df = pd.DataFrame({'run_no': flagged_runs, 'negative_outlier_count':
    negative_outlier_counts})

# Create DataFrame for results
results_df = pd.DataFrame({
    'run_no': flagged_runs,
    'variance': run_variance.loc[flagged_runs],
    'negative_outlier_count': counts_df['negative_outlier_count']
})

# Sort by negative_outlier_count descending and take top 10
results_df = results_df.sort_values(by='negative_outlier_count', ascending=
    False).head(10)

# Calculate total negative outliers across all flagged runs
total_negative_outliers = negative_outlier_counts.sum()

# Prepare output text
output_lines = []
output_lines.append("##_Flagged_Outlier_Runs\n")
output_lines.append("|_run_no_|_variance_|_negative_outlier_count_|")
output_lines.append("|-----|-----|-----|")

```

```

for index, row in results_df.iterrows():
    output_lines.append(f"|_{row['run_no']}|_{row['variance']:.2f}|_{row['negative_outlier_count']}|")

output_lines.append("")
output_lines.append(f"##_Total_Negative_'volume_ml'_Outliers_Across_Flagged_Runs")
output_lines.append(f"**Count:**_{total_negative_outliers}")

# Write to markdown file
output_file = os.path.join(output_path, 'outlier_analysis_report.md')
with open(output_file, 'w') as f:
    f.write('\n'.join(output_lines))

print(f"Report_saved_to_{output_file}")
###

```

Listing 3: Code_3